



**Politechnika
Śląska**

WYDZIAŁ AUTOMATYKI, ELEKTRONIKI I INFORMATYKI

**Laboratorium
System on Chip**

Sprawozdanie

Projekt

Zegar Dot Matrix

Autor: **Bartosz Faruga**

Rok akademicki 2025/2026, semestr 1

Gliwice, 22 czerwca 2026

1 Idea projektu

Celem projektu było zbudowanie cyfrowego zegara z funkcją budzika, którego elementem wyświetlającym jest matryca diod LED typu *dot-matrix* złożona z czterech kaskadowo połączonych modułów 8×8 sterowanych układami MAX7219. Całość zrealizowano jako system SoC (*System on Chip*) na układzie Xilinx Zynq-7020 (płyta ZedBoard), wykorzystując oba podsystemy tego układu:

- **PS** (*Processing System*) – rdzeń ARM Cortex-A9, na którym działa oprogramowanie sterujące zegara (logika aplikacji, zegar czasu rzeczywistego, obsługa klawiatury);
- **PL** (*Programmable Logic*) – część FPGA, w której zaimplementowano sprzętowy dekodery znaków ASCII oraz silnik szeregowej transmisji danych do układów MAX7219.

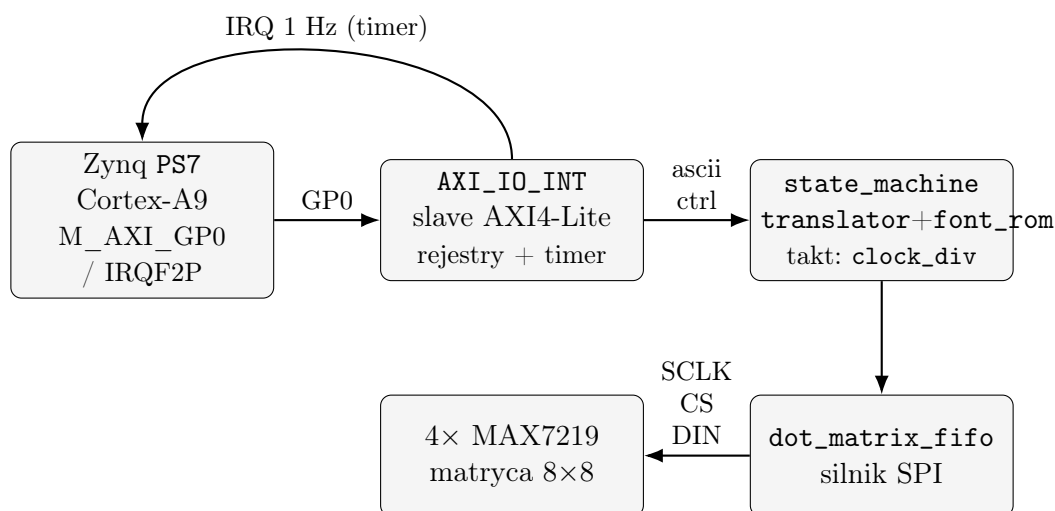
Podział zadań pomiędzy oba podsystemy wynika wprost z wymagań projektu i jest zestawiony w tabeli 1. Część czasowo-krytyczna i powtarzalna (generowanie ramek szeregowych, dekodowanie ASCII→piksele) trafiła do logiki programowalnej, natomiast logika sterująca o charakterze sekwencyjnym (tryby pracy, nastawianie czasu, antydrżaniowa obsługa przycisków, RTC) – do oprogramowania rdzenia ARM. Silnik szeregowy MAX7219 został wstępnie uruchomiony na osobnej, tańszej płycie prototypowej Tang Nano 9K (FPGA Gowin GW1NR-9), a dopiero po weryfikacji przeniesiony do układu Zynq.

Tabela 1: Wymagania projektowe i miejsce ich realizacji.

Wymaganie	Domena	Realizacja
Dekodowanie ASCII na dane MAX7219	PL	<code>font_rom</code> + <code>translator</code>
Szeregowa transmisja na matrycę	PL	<code>dot_matrix_fifo</code> (silnik SPI)
Transpozycja kolumny→wiersz	PL	wysyłanie po wierszach (rejstry cyfr)
Miganie wybranego znaku	PL/PS	rejestr <code>blink</code> + wygaszanie w firmware
Przerwania RTC	PL/PS	timer z linią IRQ + ISR
Antydrżania i autopowtarzanie	PS	<code>keyboard.h</code>
Zegar czasu rzeczywistego	PS	<code>rtc.h</code> (przerwanie 1 Hz)
Tryby pracy: czas / alarm	PS	<code>clock_app.h</code>

2 Struktura

System składa się z rdzenia ARM (PS) komunikującego się z logiką programowalną (PL) poprzez magistralę AXI4-Lite. Całość spięto w jednym module najwyższego poziomu ZED_IRQ, w którym rdzeń PS7 jest *instancjonowany bezpośrednio w Verilogu* (bez użycia integratora bloków), a jego port nadrzędny M_AXI_GP0 dołączono wprost do autorskiego slave'a AXI4-Lite AXI_IO_INT. Procesor zapisuje w ten sposób do rejestrów peryferium zakodowaną w ASCII treść do wyświetlenia oraz rozkazy sterujące; logika dekoduje te dane i steruje fizycznie matrycą LED, a zwrótnie zgłasza do procesora przerwanie timera. Ogólną architekturę przedstawia rys. 1.



Rysunek 1: Architektura SoC: PS7→AXI_IO_INT→ state_machine→matryca oraz zwrotna linia przerwania timera.

2.1 PS

Oprogramowanie rdzenia ARM napisano w języku C w konwencji *header-only* (każdy moduł to samodzielny nagłówek dołączany w jednej jednostce kompilacji). Strukturę kodu i odpowiedzialność modułów zestawiono w tabeli 2.

Tabela 2: Moduły oprogramowania PS.

Plik	Zadanie
zybo_int.c	punkt wejścia: inicjalizacja GIC i podsystemów, pętla główna
config.h	mapa rejestrów sprzętowych i stałe konfiguracyjne
rtc.h	zegar czasu rzeczywistego oparty o przerwanie 1 Hz
keyboard.h	antydrżania i autopowtarzanie 5 przycisków
display.h	sterownik matrycy MAX7219 ponad rejestrami AXI
clock_app.h	automat stanów interfejsu użytkownika (tryby, alarm)

Program nie używa systemu operacyjnego – działa w modelu *bare-metal*. Po inicjaliza-

cji kontrolera przerw (GIC), timera RTC, klawiatury i wyświetlacza wykonywana jest nieskończona pętla odpytująca (*poll loop*) o stałym okresie `POLL_MS = 10 ms` (listing 1). W każdej iteracji próbkowane są przyciski, obsługiwane jest zdarzenie sekundowe generowane przez przerwanie RTC oraz odświeżany jest stan wyświetlacza.

Listing 1: Pętla główna (`zybo_int.c`).

```
1 for (;;) {
2     app_handle_buttons(kb_scan());           // antydrżanie + autorepeat
3     if (rtc_take_sec_event())                 // flaga ustawiana w ISR
4         timera
5         app_second_tick();                     // dopasowanie alarmu co 1
6         s
7         app_refresh();                         // miganie + render na
8         matrycy
9         usleep(POLL_MS * 1000);
10 }
```

Sam zegar realizowany jest przez przerwanie sprzętowego timera o częstotliwości 1 Hz (wektor GIC nr 61). Procedura obsługi przerwania (listing 2) kasuje flagę przerwania, inkrementuje liczniki sekund/minut/godzin i ustawia flagę `rtc_sec_event`, którą pętla główna konsumuje raz na sekundę. Odczyt i zapis czasu wykonywany jest przy chwilowo zamaskowanym przerwaniu, co eliminuje ryzyko odczytu niespójnego (*tearing*).

Listing 2: Procedura obsługi przerwania RTC (`rtc.h`).

```
1 static void rtc_isr(void *cb) {
2     rtc_tm->tmCTRL = (1u << TM_INT) | (1u << TM_RUN); // skasuj
3     IRQ, timer dalej liczy
4     if (++rtc_ss >= 60) { rtc_ss = 0;
5         if (++rtc_mm >= 60) { rtc_mm = 0;
6             if (++rtc_hh >= 24) rtc_hh = 0; } }
7     rtc_sec_event = 1;
8 }
```

Obsługa klawiatury (`keyboard.h`) realizuje dwa wymagane mechanizmy. **Antydrżanie**: stan przycisku zmienia się dopiero po `DEBOUNCE_TICKS = 3` kolejnych identycznych próbkach (30 ms). **Autopowtarzanie**: przytrzymanie przycisku przez `REPEAT_DELAY = 500 ms` uruchamia powtórzenie co `REPEAT_PERIOD = 120 ms`. Każdy przycisk ma własny, niezależny stan (struktura `Btn`), a funkcja `kb_scan()` zwraca maskę zdarzeń do dalszego przetworzenia przez automat aplikacji.

2.2 PL

Część logiki programowalnej opisano w języku Verilog. Hierarchię modułów oraz ich rolę zestawiono w tabeli 3. Modułem nadrzędnym jest `state_machine` – automat sterujący, który na podstawie rejestru rozkazów wykonuje sekwencje wysyłane do matrycy. Cała logika sterowania matrycą taktowana jest wolniejszym sygnałem z `clock_div` (100 MHz dzielone przez 10), dzięki czemu wynikowy zegar szeregowy SCLK nie przekracza dopuszczalnych dla MAX7219 10 MHz.

Tabela 3: Moduły logiki programowalnej (PL).

Moduł	Zadanie
<code>top / state_machine</code>	automat główny: inicjalizacja, jasność, render ASCII
<code>translator</code>	dla danego wiersza pobiera piksele 4 znaków i składa ramkę
<code>font_rom</code>	ROM czcionki: kod ASCII (7 b) → bitmapa 8×8
<code>dot_matrix_fifo</code>	silnik szeregowy (SPI) sterujący MAX7219
<code>clock_div</code>	podział 100 MHz / 10 – takt <code>state_machine</code> (SCLK ≤ 10 MHz)

Silnik szeregowy. Moduł `dot_matrix_fifo` implementuje protokół MAX7219: 16-bitowe słowa wysyłane bitem najstarszym (MSB) naprzód, gdzie DIN próbkowane jest na zboczu narastającym SCLK. Dla $N = 4$ kaskadowo połączonych układów wysyłane jest $N \times 16 = 64$ bity przy aktywnym (niskim) sygnale CS/LOAD, po czym zbocze narastające CS zatrzymuje wszystkie słowa jednocześnie. Automat (stany IDLE/SHIFT/LATCH) sygnalizuje zajętość linią `busy`; dostępne jest też wejście `abort` pozwalające natychmiast przerwać bieżącą ramkę. Format słowa sterującego MAX7219 to: bity [11:8] – adres rejestru, [7:0] – dane. Najważniejsze rozkazy konfiguracyjne zestawia tabela 4.

Tabela 4: Słowa konfiguracyjne MAX7219 użyte w sekwencji inicjalizacji.

Słowo	Rejestr	Znaczenie
0x0C01	shutdown	wyjście ze stanu uśpienia (praca normalna)
0x0B07	scan-limit	skanowanie wszystkich 8 wierszy
0x0900	decode mode	brak dekodowania (surowe segmenty)
0x0A0n	intensity	jasność (sterowana z PS, poziomy 0–3)

Dekodowanie i transpozycja. Wymaganie „transpozycji z zapisu kolumnowego na wyświetlanie wierszowe” zrealizowano przez wysyłanie obrazu *wiersz po wierszu*. Dla wiersza r moduł `translator` pobiera z `font_rom` 8-bitowy wzór tego wiersza dla każdego z 4 znaków i składa ramkę 4×16 b, w której każde słowo adresuje rejestr cyfry MAX7219 ($r+1$, bo rejestry numerowane są 1–8) – listing 3. Automat `state_machine` powtarza to dla wszystkich 8 wierszy, dając kompletne odświeżenie wyświetlacza w 8 ramkach szeregowych.

Listing 3: Złożenie słowa ramki w module translator.

```

1 // rejestry cyfr MAX7219 numerowane 1..8, wiec adres = row_idx + 1
2 row_packet[(N-1-idx)*16 +: 16] <= {4'b0000, ({1'b0,row_idx} + 4'd1)
    , row_data};

```

2.3 AXI Lite

Komunikację PS↔PL zrealizowano przez magistralę AXI4-Lite, lecz – inaczej niż w typowym przepływie z integratorem bloków – bez gotowych rdzeni IP. Rdzeń PS7 jest instancjonowany ręcznie w module ZED_IRQ, a jego port nadrzędny M_AXI_GP0 dołączono wprost do autorskiego slave'a AXI_IO_INT. Moduł ten zawiera kompletne automaty obsługi kanałów AXI4-Lite (zapis: W_IDLE→W_WR→W_RESP; odczyt: R_IDLE→R_DATA→R_DATA_ACK), dekoduje adres transakcji na zestaw rejestrów (tab. 5) i udostępnia logice sterującej sygnały ASCII_DATA oraz CTRL_REG. Zapis rejestru obrazu honoruje maskę bajtów WSTRB, więc procesor może modyfikować pojedyncze znaki. Wyjścia sterownika (MAX_CLK, MAX_LOAD, MAX_DIN) wyprowadzono na złącze Pmod ZedBoard (piny AA9 / AA11 / Y11).

Tabela 5: Mapa rejestrów slave'a AXI_IO_INT (przestrzeń AXI GP0).

Rejestr	Adres	Dostęp	Opis
ASCII_DATA	0x40000000	RW	4 znaki × 7 b – pamięć obrazu (z WSTRB)
CTRL_REG	0x40000004	RW	rozkaz: [7:6] op, [5] kick, [1:0] arg
LED (clr/tgl)	0x08 / 0x0C	W	dioda alarmu: zapis 1 zeruje / neguje bit
SW	0x40000020	R	przełączniki suwakowe
PB	0x40000024	R	5 przycisków (L/R/D/U/C)
TMR_Q	0x40000040	RW	wartość przeładowania (start gdy ≥256)
TMR_CTRL	0x40000044	RW	START/STOP/RUN, flagi ZD (INT)/OV
TMR_CNT	0x40000048	R	bieżący stan licznika

Mechanizm „kick”. Ponieważ rozkazy są przekazywane przez rejestr (a nie strumień danych), konieczne było rozróżnienie dwóch identycznych zapisów następujących po sobie. Rozwiązano to bitem kick (bit 5 ctrl_reg): firmware neguje go przy każdym zapisie (listing 4), a logika wykrywa zbocze tego bitu, generując jednotaktowy impuls cmd_valid. Pozwala to ponawiać ten sam rozkaz dowolną liczbę razy. Dodatkowo wejściowy ctrl_reg jest synchronizowany trzema przerzutnikami (CDC) przed użyciem, a przyjście nowego rozkazu w trakcie trwającej transmisji powoduje jej przerwanie (abort) i obsłużenie nowego polecenia.

Listing 4: Wysyłanie rozkazu z bitem „kick” (display.h).

```

1 static inline void display_cmd(uint32_t op, uint32_t arg) {
2     display_kick ^= 1u; //
    negacja przy kazdym zapisie

```

```

3     Xil_Out32(CTRL_REG, op | (display_kick << 5) | (arg & 3u));
4 }

```

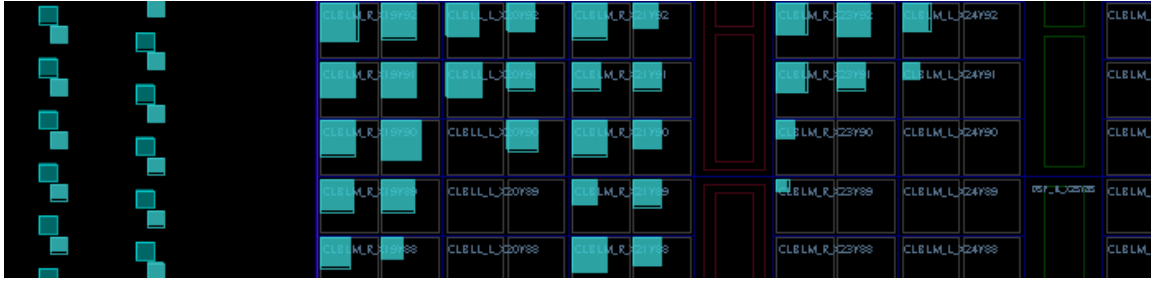
RTC i przerwania. Zegar czasu rzeczywistego opiera się o timer wbudowany w AXI_IO_INT: licznik TMR_CNT zlicza w górę od 0 do wartości TMR_Q, po czym zeruje się i ustawia flagę przejścia przez zero ZD (bit 31), będącą źródłem linii IRQ. Linia ta trafia do wejścia IRQF2P[0] rdzenia PS7, co odpowiada przerwaniu o wektorze 61 w kontrolerze GIC. Przy taktowaniu 100 MHz wartość $TMR_Q = 99\,999\,999$ daje przerwanie dokładnie raz na sekundę, a procedura obsługi kasuje flagę zapisem 1 na bit 31 rejestru sterującego. Sam timer (z flagami ZD/OV oraz warunkiem startu $TMR_Q \geq 256$) pochodzi z przykładu laboratoryjnego, rozszerzonego na potrzeby projektu o rejestry ASCII_DATA i CTRL_REG.

2.4 pamięć rom

Blok pamięci czcionki (BRAM). Moduł font_rom odwzorowuje 7-bitowy kod ASCII (znaki 0x20–0x7E) na bitmapę 8×8 pikseli (64 bity). Adres znaku jest rejestrowany (`addr <= ascii_code`), a odczyt wiersza odbywa się synchronicznie – jest to klasyczny wzorzec *pamięci synchronicznej*, dzięki któremu narzędzie syntezy odwzorowuje czcionkę na sprzętowy blok pamięci RAM (BRAM), a nie na zasoby logiczne (LUT). Wynikające stąd jednotaktowe opóźnienie odczytu jest ukryte przez stan WAIT_ROM modułu translator. Dzięki pełnemu zestawowi cyfr, liter i znaków ten sam wyświetlacz prezentuje zarówno czas (np. 12:30), jak i komunikaty tekstowe.

Pamięć obrazu. Wymaganą „pamięć obrazu z kodowaniem ASCII” oraz „bezpośredni dostęp RW do pamięci wyświetlacza” realizuje rejestr ASCII_DATA w AXI_IO_INT: firmware pakuje 4 znaki (po 7 bitów) w jedno 28-bitowe słowo makrem PACK4 i zapisuje je jednym dostępem AXI. Aby ograniczyć ruch na magistrali, `display_send()` buforuje ostatnio wysłaną zawartość i ponawia transmisję tylko przy faktycznej zmianie obrazu. W ramach eksperymentu (moduł `bram.v`) testowano też realizację pamięci obrazu jako prawdziwego bloku BRAM 256×32 b z adresowaniem, co stanowi ścieżkę rozbudowy w kierunku w pełni adresowalnej, bezpośrednio zapisywalnej pamięci wyświetlacza.

Efekt implementacji widać na widoku struktury krzemowej układu (rys. 2): narzędzie rozmieściło logikę projektu w komórkach konfigurowalnych (CLB) – zajęte przerzutniki i tablice LUT zaznaczono kolorem turkusowym – obok regularnych, pionowych kolumn dedykowanych zasobów (bloki pamięci BRAM oraz DSP). Pamięć czcionki `font_rom` trafia właśnie do takiego bloku BRAM, zamiast zajmować zasoby logiczne CLB.



Rysunek 2: Fragment macierzy układu Zynq-7020 w widoku „device” narzędzia Vivado: komórki CLB (etykiety CLB_LM_R/CLB_LM_L) z zajętymi zasobami (turkus) oraz pionowe kolumny zasobów dedykowanych (BRAM/DSP).

3 Opis działania

Inicjalizacja. Po starcie firmware wysyła rozkaz `ENABLE`, w odpowiedzi na który logika nadaje trzy ramki konfiguracyjne MAX7219 (tabela 4): wyjście z uśpienia, ustawienie skanowania 8 wierszy oraz wyłączenie dekodowania. Następnie ustawiana jest domyślna jasność.

Tryby pracy. Aplikacja (`clock_app.h`) działa jako automat trzech stanów przełączanych przyciskiem środkowym (C):

- `MODE_TIME` – wyświetlanie bieżącego czasu; góra/dół zmienia jasność, lewy włącza/wyłącza alarm;
- `MODE_SET_TIME` – nastawianie czasu; lewy/prawy wybiera pole godzin/minut, góra/dół modyfikuje wartość;
- `MODE_SET_ALARM` – nastawianie czasu alarmu (analogicznie).

Wyjście z trybu `SET_TIME` zatwierdza nastawiony czas w RTC.

Miganie nastawianego pola. Podczas nastawiania edytowane pole (godziny lub minuty) miga z okresem 250 ms. Realizowane jest to programowo: co `BLINK_TICKS` iteracji pętli zmienia się faza migania i w fazie wygaszonej odpowiednie znaki zastępowane są spacją przed wysłaniem na wyświetlacz. Sprzętowo dostępny jest też rejestr „pozycji do migania” (rozkaz `BLINK`), zgodny z wymaganiem rejestru pozycji znaku.

Alarm. Raz na sekundę (`app_second_tick`) sprawdzane jest dopasowanie bieżącego czasu do nastawionego alarmu. Aktywny alarm powoduje miganie całego wyświetlacza oraz diody sygnalizacyjnej; dowolny przycisk wycisza alarm.

Potok renderowania. Po zapisaniu nowej treści (`ASCII_DATA`) i rozkazie `ASCII`, automat PL przechodzi przez 8 wierszy: dla każdego z nich uruchamia `translator` (pobranie pikseli i złożenie ramki), oczekuje na gotowość, a następnie zleca silnikowi szeregowemu wysłanie ramki do matrycy. Mechanizm `abort` gwarantuje, że nowy rozkaz z PS przerywa trwającą transmisję, dzięki czemu obraz reaguje natychmiast na działania użytkownika.

4 Wnioski

- Zrealizowano kompletny system SoC zegara z budzikiem, poprawnie dzieląc zadania między rdzeń ARM (logika aplikacji, RTC, klawiatura) a logikę programowalną (dekodowanie ASCII i transmisja szeregową), co jest naturalnym i efektywnym podziałem w architekturze Zynq.
- Ręczne napisanie slave'a AXI4-Lite (AXI_IO_INT) i bezpośrednie instancjonowanie PS7 dało pełną kontrolę nad mapą rejestrów i linią przerwania; kluczowy okazał się mechanizm „kick” oraz synchronizacja CDC, które zapewniły niezawodne przekazywanie rozkazów mimo braku dedykowanego protokołu strumieniowego.
- Wstępne uruchomienie silnika MAX7219 na płycie Tang Nano 9K znacząco przyspieszyło prace – protokół szeregowy zweryfikowano niezależnie od złożoności toru AXI, a na Zynq przeniesiono już działający moduł.
- Przerwanie sprzętowego timera (1 Hz) zapewniło dokładny i niezależny od obciążenia pętli głównej pomiar czasu, a maskowanie przerwania przy odczycie czasu wyeliminowało niespójności danych.
- Naturalnym kierunkiem rozbudowy jest zastąpienie rejestru obrazu pełną, adresowalną pamięcią BRAM (prototyp `bram.v`) oraz przeniesienie migania w całości do sprzętu z wykorzystaniem istniejącego rejestru pozycji.